Case Trainer — Build Specification for Claude Code

A Flutter Android app that examines a data-science student on 50 real business problems (Aviation, Formula 1, Football, Finance), pre-solved offline on real public data. Owner: Awadhi · Target device: Samsung Galaxy S21 / S21+ · Version 1.0 · 5 November 2026

Read this first, Claude Code. This PDF is the single source of truth for the project. Section 2 contains hard rules that override anything else, including your own assumptions. Anything marked **UNVERIFIED** must be checked (open the URL, load the file, print the schema) before use, and the check recorded in provenance.json. You must never invent data, numbers, column names, package APIs, or citations. If something is unknown, stop and say so.

Contents

- 1. What we are building (product + examiner loop)
- 2. Hard rules: Do's and Don'ts (anti-hallucination)
- 3. Target device and technical architecture (S21)
- 4. Repository layout and content bundle format
- 5. The examiner: grading engine, 3 attempts, 85% threshold
- 6. Reference audit for all 50 problems (Gemini's citations checked)
- 7. Data availability and tiers for all 50 problems
- 8. Tier A build order (what we build first)
- 9. Solution blueprints for the Tier A problems
- 10. Step-by-step build prompts (paste into Claude Code in order)
- 11. Verification register (what still needs checking)
- 12. Sources

1. What we are building

Owner. Awadhi, MSc Data Science student (City St George's, University of London). Strong in Python and web development; wants to train the *thinking* part of data science: turning a business problem into questions, reading charts, choosing methods, and explaining results in his own words. He practises on his Android phone and later turns finished cases into portfolio projects on his PC.

Core idea. Every problem is **already solved** by an offline Python pipeline on a PC using **real public data**. The pipeline writes the cleaned data, every chart's data, every number, the model answers and the rubrics into a content bundle. The Flutter app on the phone does not compute results: it **presents and examines**. This makes the app fast, offline, reliable on an older phone, and impossible for the app to show a made-up number.

The case flow (every case follows this)

Phase	What the user does	How the app examines
1 Brief	Reads the business problem: decision-maker, decision, money at stake	Comprehension MCQ
2 Frame	Chooses the right question, KPIs, writes hypotheses	MCQ + KPI builder + free-text (graded)
3 Data	Meets the real dataset: schema, units, missing values, cleaning code	Predict-the-output, fill-the-blank, fix-the-bug (outputs precomputed)
4 Explore	Interacts with charts (zoom, tap points, toggle series)	Explains each chart in own words (graded, 3 attempts, 85%)
5 Model	Sees the method and results (regression, test, classifier...)	Picks the right method, interprets numbers in own words
6 Decide	Writes the business recommendation memo	Memo rubric incl. red flags (causation, certainty)
7 Debrief	Concepts learned, mistakes made, 30-second interview summary	Spaced-review cards scheduled

The examiner loop (owner's requirement)

- The user explains in his own words. The app scores the explanation against a rubric (0–100%).
- **Pass = score ≥ 85% and no critical red flag.**
- Attempt 1 fails → show which rubric areas are missing (not the answer) plus a structure template for how to explain well.
- Attempt 2 fails → targeted hints for each missing point and an explanation of any red flag, plus guidance on the analysis step needed.
- Attempt 3 fails → reveal the model answer with each rubric point highlighted, explain why, and schedule the item for spaced re-test.
- Every attempt, score and hint is stored on the phone for the Progress screen.

2. Hard rules: Do's and Don'ts for Claude Code

These rules override everything else in this document and any of your own defaults. Anthropic's guidance: keep CLAUDE.md short, and prefer deterministic checks (tests, typechecks, CI) over instructions; allow 'I don't know'; verify claims with sources and retract what cannot be supported. (code.claude.com/docs/en/best-practices; docs.claude.com ... /reduce-hallucinations)

DO

- Use **plan mode** for every multi-file change. Show the plan, wait for approval.
- Load every raw file and **print its schema** (columns, dtypes, head, row count) before writing code that uses it.
- Write one **provenance.json** entry per raw file: source URL, retrieval timestamp, SHA-256, licence/terms note, row count, columns, and every transformation applied (with counts of rows dropped and why).
- Have the pipeline write **expected_results.json** per case. Every number shown in the app must be a key in it. A pytest asserts each value; the content build fails if a referenced key is missing.

- Mark every external claim as **VERIFIED** (URL opened, quote stored) or **UNVERIFIED**. Show the badge in the app.
- Use time-based train/test splits for time-ordered data. Fix random seeds. Pin package versions (lockfiles for Python and Flutter).
- Write rubric red flags from the misconception lists in Section 9.
- Check package names, versions and APIs on pub.dev / PyPI / official docs before using them. If a package is discontinued, say so and propose an alternative.
- Measure performance on the real device (Galaxy S21) before claiming it is fast enough.
- Commit small, with clear messages. Run all tests before saying a task is done.

DON'T

- **Don't** type numbers into Dart, JSON content, Markdown or rubric files by hand. Numbers come only from the pipeline.
- **Don't** invent dataset columns, series IDs, file names or API endpoints. Verify them.
- **Don't** cite Gemini's original references, or any source not in sources.json with an opened URL.
- **Don't** use synthetic or simulated data in any case. This version builds **only Tier A cases with real data**. If real data cannot be obtained, stop and report; do not substitute.
- **Don't** silently drop outliers or missing values. Log what was removed and why.
- **Don't** let any LLM (local or cloud) decide pass/fail or change stored numbers. The deterministic grader decides; LLMs may only rephrase feedback.
- **Don't** redistribute data beyond its terms. StatsBomb data requires credit and logo; LOBSTER samples are for research use; check Yahoo/Kaggle terms before bundling.
- **Don't** build features from a later prompt. Do only what the current prompt asks.
- **Don't** claim causation in model answers unless the design supports it; always state limitations.

3. Target device and technical architecture

Device

Item	Value	Implication
Phone	Samsung Galaxy S21 / S21+ (UK models use Exynos 2100; US models Snapdragon 888)	Test on the actual phone; no published LLM benchmarks exist for these chips
RAM	8 GB	Keep app working set well under 2 GB; any local LLM must be optional and on-demand
OS	Android 13–15 (One UI 5–7). Owner to confirm in Settings > About phone > Software information	minSdk 26 is safe; set targetSdk to latest stable; native libs must support 16 KB pages for Android 15
ABI	arm64-v8a	Build with --split-per-abi and install only the arm64 APK
Size budget	Up to ~3 GB allowed by owner	Core app + all content target < 500 MB; optional models downloaded/copied separately

Architecture decision: precompute on PC, examine on phone

```
PC (Python pipeline)                          Phone (Flutter app, offline)
------------------------------                --------------------------------
download real data  -> raw/ + provenance.json   content.sqlite (read-only, bundled)
clean + analyse     -> expected_results.json    charts/*.json  (interactive charts)
charts              -> charts/*.json + *.png    charts/*.png   (fallback/exam stimulus)
rubrics + answers   -> case.yaml                models/embedder (ONNX, ~23-90 MB)
pytest (asserts)    -> build_content.py  ---->  progress.sqlite (writable: attempts,
calibration set     -> grader tests                              scores, reviews)
```

Recommended stack (verify versions on pub.dev / PyPI before use)

Layer	Choice	Notes / status
Pipeline	Python 3.11+, pandas, numpy, statsmodels, scipy, scikit-learn, matplotlib, fastf1, pytest	FastF1 requires Python ≥ 3.10 (verified, GitHub). Pin with a lockfile.
App framework	Flutter (stable), Dart	State: Riverpod (or Bloc). Navigation: go_router. Versions UNVERIFIED — check.
Charts	Syncfusion Flutter Charts (primary), fl_chart (fallback)	Syncfusion Community Licence is free for individuals/orgs under US$1M revenue, ≤5 devs, ≤10 employees, after registration (verified). Owner must register. fl_chart licence UNVERIFIED.
Chart types needed	line (multi-series, date axis), scatter (+ fitted line), histogram, box plot, bar/stacked bar, heatmap, shaded regions (recessions)	Downsample in pipeline (LTTB) to < 2,000 points per series; full data kept separately
Storage	SQLite: drift or sqflite	content.sqlite read-only (copied from assets on first run), progress.sqlite writable
Text grading	Sentence embeddings on-device: all-MiniLM-L6-v2 ONNX (384-dim; arm64 int8 file ~23 MB) via ONNX Runtime, or MediaPipe Text Embedder via platform channel	MiniLM sizes from Hugging Face (verified). Flutter ONNX plugin choice UNVERIFIED; WordPiece tokenizer must be implemented (Dart or native).
Optional local LLM	Gemma 3 1B int4 (~0.5–0.7 GB) via flutter_gemma or its successor	pub.dev marks flutter_gemma discontinued, replaced by flutter_edge_ai — verify. Used ONLY to rephrase feedback. Peak RAM ~1 GB on S24 Ultra; S21 untested.
Optional online	Claude API for richer feedback when online	User's own key stored on device; never decides pass/fail.
Code on device	v1: NO on-device Python. Code tasks use outputs precomputed by the pipeline	v2 option: serious_python (bundles CPython; numpy/pandas). matplotlib/statsmodels on Android ARM64 UNVERIFIED.

Why no Python on the phone in v1: the owner wants to visualise graphs and explain them reliably on an older phone. Interactive charts rendered natively from precomputed data are smoother and fully offline. Code learning still happens: he reads real pipeline code, predicts outputs, fills blanks and fixes bugs, and the app compares his answer to the real output the pipeline captured.

4. Repository layout and content bundle

```
case-trainer/
  CLAUDE.md                 short project rules (points to this PDF)
  docs/spec.pdf             THIS document
  docs/source/              Gemini's original PDF (reference only, citations untrusted)
  pipeline/
    common/                 provenance.py, charts.py (JSON + PNG export, LTTB), io.py
    cases/p50_yield_curve/  download.py, clean.py, analyse.py, content.yaml, test_case.py
    cases/p34_home_adv/ ...
    calibration/<case>/     hand-labelled answers for grader calibration
    build_content.py        validates everything -> app/assets/content/
  app/                      Flutter project
    assets/content/         content.sqlite, charts/*.json, charts/*.png
    assets/models/          embedder.onnx, vocab.txt
    lib/ engine/ grading/ ui/ data/
  sources.json              every citation: title, authors, year, URL, verified_on, quote
```

Per-case content.yaml (authored by Claude Code, numbers referenced by key)

```
id: p50_yield_curve
number: 50
domain: finance
title: Does yield-curve inversion predict US recessions?
tier: A
decision_maker: Portfolio manager at a UK asset manager
decision: Reduce equity risk when the curve inverts?
sources: [estrella_mishkin_1996, nyfed_ycfaq]        # keys in sources.json
datasets: [fred_T10Y2Y, fred_T10Y3M, fred_USREC]     # keys in provenance.json
steps:
  - id: s4_chart1
    type: chart_explain
    chart: charts/p50_spread_recessions.json
    prompt: Explain what this chart shows and what it does NOT show.
    scaffold: chart_6q           # What plotted / Pattern / Strength / Anomalies / Can't show / So what
```

```
    rubric:
      points:
        - id: axes
          weight: 15
          paraphrases: ["the line is the 10-year minus 2-year Treasury yield spread in percentage points
over time",
                        "shaded areas are NBER recession periods"]
          keywords: [[spread, difference], [recession, shaded]]
        - id: lead
          weight: 30
          paraphrases: ["the spread usually turns negative some months before recessions start"]
          keywords: [[negative, invert, below zero], [before, ahead, lead]]
          numeric: {key: median_lead_months, tolerance: 3}   # value from expected_results.json
        ...
      red_flags:
        - id: causation
          critical: true
          paraphrases: ["inversion causes recessions"]
          feedback: The chart shows timing, not cause. Say 'precedes' or 'is associated with'.
    hints: [ "...", "..." ]
    model_answer: "{{text with {expected.median_lead_months} placeholders}}"
```

build_content.py fails if: a placeholder key is missing from expected_results.json; a source key is missing or unverified; a chart file is missing; a rubric's weights do not sum to 100; or the case has no calibration set.

5. The examiner: grading engine (3 attempts, 85%)

Rubric per question

- 4–7 **key points**, each with a weight, 3–6 reference paraphrases, required concept groups (keywords + synonyms), and an optional numeric anchor (a key in expected_results.json with a tolerance).
- 3–6 **red flags** (misconceptions), each with paraphrases, a penalty, a 'critical' flag, and targeted feedback.

Scoring algorithm (deterministic, offline)

- 1. Split the answer into sentences; embed each sentence and each paraphrase (cosine similarity).
- 2. Semantic score per key point = max cosine over (sentence, paraphrase) pairs, mapped to coverage 0–1 with a logistic curve fitted on calibration data (never raw cosine thresholds).
- 3. Keyword score per point = fraction of concept groups present (lower-case, lemmatised, synonym lists). Point coverage = 0.6 × semantic + 0.4 × keyword (weights tuned during calibration).
- 4. Numeric anchors: extract numbers from the answer; credit only if within tolerance of the pipeline value.
- 5. Red flags: if matched above its calibrated threshold, subtract the penalty. A negation check stops 'it is not causal' triggering the causation flag.
- 6. Total = Σ(weight × coverage) − penalties, clipped to 0–100. **Pass if total ≥ 85 and no critical red flag.**

Calibrating the 85% threshold (required before a case ships)

- Per question, write 30–50 example answers (strong, partial, wrong, red-flag) and label each 'acceptable' or 'not acceptable' by hand. The owner should label or review at least some himself.
- Fit the mapping curves; tune weights so acceptable answers score ≥ 85 and the false-pass rate is below 5%.
- Report agreement with human labels (Cohen's κ). Store the set and metrics; re-run as tests whenever the model or rubric changes.

Feedback on each attempt

Attempt	If it fails, the app shows
1	Score; which rubric areas are missing (names only, not answers); the explanation template for this step type (below); one sentence on what analysis to look at again
2	Targeted hint for each missing point (e.g. 'Look at what happens to the spread about 12 months before each shaded bar'); explanation of any red flag triggered; the analysis step to redo (e.g. 'compare the daily changes chart, not the levels chart')
3	Model answer with each rubric point highlighted and linked to the evidence (chart region or number), why each point matters, and the item is scheduled for spaced re-test (1, 3, 7, 21 days)

Explanation templates the app teaches

Step type	Template
Chart	1) What is plotted (axes, units, period) → 2) Pattern (direction, shape, seasonality) → 3) Strength/size (a number: r, slope, %) → 4) Anomalies (outliers, gaps, regime changes) → 5) What it can't show (causation, confounders, spurious trend) → 6) So what for the business
Result / model output	Claim – Evidence – Reasoning: claim in one sentence; the number(s) that support it; why that evidence supports the claim, plus one limitation
Business memo	Recommendation (with a number/range) → Evidence (2–3 numbers) → Risk left over → Limitations → Next step

Learning-science basis: Dunlosky et al. (2013) rate practice testing and distributed practice as high utility, and self-explanation and interleaving as moderate utility — supporting graded self-explanations, spaced re-tests and mixing domains in the build order. (Psychological Science in the Public Interest 14(1), 4–58.)

6. Reference audit: Gemini's 50 citations

Status: **PARTIAL** = the organisation/journal is real, but the specific dated document Gemini named was not found. **NOT FOUND** = no matching publication identified; treat as fabricated. No Gemini citation was confirmed as written. **Never show Gemini's citations in the app.** Use only the real references listed, and only after re-opening them.

#	Problem	Gemini citation	Status	Real references (re-open before use)
1	Jet fuel hedging	IATA Fuel Monitor 2024	PARTIAL	Turner & Lim (2015) J. Air Transport Mgmt, pmc.ncbi.nlm.nih.gov/articles/PMC7147842 · TRF (2016) Airline Fuel Hedging, trforum.org/wp-content/uploads/2016/10/2016v55n1_03_AirlineFuelHedging.pdf
2	Airfare elasticity	Brueckner 2004 MIT Press book	NOT FOUND	None verified
3	Delay propagation	Journal of Air Transport 2021	NOT FOUND	BTS On-Time Statistics & Delay Causes, transtats.bts.gov/ot_delay/ot_delaycause1.asp
4	APU maintenance	FAA Safety Report 2023	PARTIAL	None verified
5	Baggage bottlenecks	ACI World 2022	PARTIAL	None verified
6	Belly cargo	Boeing Aero 2024	PARTIAL	None verified
7	Pilot fatigue	Flight Safety Foundation 2023	PARTIAL	None verified
8	Slot valuation	Eurocontrol 2024	PARTIAL	None verified
9	CORSIA pricing	ICAO 2025	PARTIAL	None verified
10	Gate assignment	J. Air Transport Mgmt 2022	PARTIAL	None verified
11	De-icing inventory	FAA Weather Ops 2023	NOT FOUND	None verified
12	Loyalty liability	Airline Finance News 2024	NOT FOUND	Airline 10-K filings on SEC EDGAR (UNVERIFIED)
13	Tyre degradation	Pirelli Motorsport 2024	PARTIAL	FastF1 docs, docs.fastf1.dev/core.html
14	Undercut/overcut	Motorsport Engineering 2023	NOT FOUND	Jolpica-F1 docs, github.com/jolpica/jolpica-f1/blob/main/docs/README.md
15	Fuel load penalty	ScarbsF1 Analysis 2024	NOT FOUND	None verified
16	Wing vs DRS	FIA Technical Review 2024	NOT FOUND	None verified
17	MGU-K	Mercedes HPP 2023	PARTIAL	None verified
18	Safety car	Race Strategy Group 2024	NOT FOUND	FastF1 track_status, docs.fastf1.dev/core.html
19	Wind tunnel	Autosport Engineering 2024	NOT FOUND	None verified
20	Cost cap	FIA Regulations 2024	PARTIAL	None verified (regulations exist; version not opened)
21	Brake ducts	Brembo Tech 2023	PARTIAL	None verified
22	Launch traction	F1 Telemetry Data 2024	NOT FOUND	Jolpica results + laps
23	Pit crew fatigue	Red Bull Racing Science 2024	NOT FOUND	Jolpica /pitstops/
24	Gearbox reliability	Transmission Engineering 2023	NOT FOUND	Jolpica /status/
25	UCL exit share drop	FT Football 2024	PARTIAL	None verified (event-study method refs needed)
26	xG valuation	StatsBomb Research 2024	PARTIAL	StatsBomb Open Data, github.com/statsbomb/open-data
27	Pressing vs injury	FIFA Medical Report 2024	PARTIAL	None verified
28	Rotation	UEFA Elite Club Study 2024	PARTIAL	None verified
29	Set pieces	Opta Analyst 2024	PARTIAL	None verified
30	Ticket pricing	Football Business Journal 2023	NOT FOUND	None verified
31	Academy ROI	CIES 2024	PARTIAL	None verified

#	Problem	Gemini citation	Status	Real references (re-open before use)
32	Sponsorship exposure	Nielsen Sports 2024	PARTIAL	None verified
33	New-manager bounce	Sloan Sports Analytics 2023	PARTIAL	Erasmus Univ. Rotterdam release (regression to the mean), eur.nl/en/news/irrational-managerial-changes-remain-norm-europes-top-football-leagues · Pinnacle summary of ter Weel's Eredivisie study
34	Home advantage	J. Sports Economics 2022	PARTIAL	Systematic review 'The cauldron has cooled down', pmc.ncbi.nlm.nih.gov/articles/PMC8724651 · Leitner & Richlan (2021) Frontiers, frontiersin.org/journals/sports-and-active-living/articles/10.3389/fspor.2021.720488/full
35	Broadcast revenue	Deloitte Money League 2024	PARTIAL	None verified (series real; edition not opened)
36	Deadline day	Transfermarkt Analytics 2024	NOT FOUND	None verified
37	Fan tokens	Blockchain Sports Review 2024	NOT FOUND	None verified
38	Order-book imbalance	J. Financial Data Science 2024	PARTIAL	LOBSTER technical report, data.lobsterdata.com/info/docs/LobsterReport.pdf
39	Cap rates	Real Estate Economics 2024	PARTIAL	None verified
40	Credit scoring	Global Fintech Review 2024	NOT FOUND	None verified
41	Pairs trading	Adv. in Financial ML 2023 Wiley	PARTIAL (wrong year)	López de Prado, Advances in Financial Machine Learning, Wiley, Feb 2018, ISBN 978-1-119-48208-6
42	Mortgage prepayment	MBS Portfolio Mgmt 2024	NOT FOUND	None verified
43	ESG greenwashing	Sustainable Finance Journal 2024	NOT FOUND	None verified
44	VC unicorn	Venture Capital Journal 2024	PARTIAL	None verified
45	Cat bonds	Swiss Re sigma 2024	PARTIAL	None verified
46	FX carry	FX Week Analytics 2023	PARTIAL	None verified
47	Bond liquidity	Fixed Income Journal 2024	NOT FOUND	None verified
48	Vol surface	J. Derivatives 2024	PARTIAL	None verified
49	PropTech rents	Urban Economics Review 2024	NOT FOUND	None verified
50	Yield curve recession	FRED 2024	PARTIAL (data service)	Estrella & Mishkin (1996) NY Fed Current Issues 2(7), newyorkfed.org/research/current_issues/ci2-7.html · NY Fed yield curve FAQ, newyorkfed.org/research/capital_markets/ycfaq · FEDS Note 2018 (federalreserve.gov) · Wright (2006) FEDS 2006-07

Important correction for P50: the NY Fed model uses the **10-year minus 3-month** spread, not 2s10s. Teach both: 2s10s as the question, 10y–3m as the literature benchmark.

7. Data availability and tiers

A = real free data answers it directly. **A*** = real free data, plus a small hand-compiled table (e.g. event dates) with a source URL per row. **B** = a real proxy answers a reframed question (later version). **C** = no public data — excluded. **v1 builds only A and A* cases.**

#	Tier	Dataset(s)	Note / reframe
1	A	FRED DJFUELUSGULF (US Gulf jet fuel, $/gallon, daily, source EIA, verified) + DCOILBRENTEU (Brent, $/barrel, daily, verified)	Spot only; futures not free
2	B	BTS DB1B fares (UNVERIFIED)	Fares vs distance/competition; no booking-window data
3	A	BTS Reporting Carrier On-Time Performance 1987–present, delay-cause fields (verified)	Large monthly files: pre-aggregate
4	B	NASA C-MAPSS turbofan (UNVERIFIED)	RUL proxy for APU
5	C	—	Exclude
6	B	BTS T-100 freight (UNVERIFIED)	Freight per departure vs distance
7	C	—	Exclude
8	C	—	Exclude
9	C	—	Exclude
10	B	BTS On-Time taxi-out	Taxi-out vs departures per hour
11	C	—	Exclude
12	B	SEC EDGAR 10-Ks (UNVERIFIED)	Loyalty deferred revenue trend
13	A	FastF1 laps (verified; Python ≥3.10). Field names (Compound, TyreLife) to verify	Personal use; check data terms
14	A	FastF1 laps + Jolpica laps/pitstops (verified docs)	Respect rate limits
15	B	FastF1 laps	Lap time vs lap number in a stint (fuel and tyre confounded)
16	B	FastF1 speed traps (UNVERIFIED)	Top speed vs sector times
17	C	—	Exclude
18	A	FastF1 track_status (verified) + Jolpica	SC/VSC rate per circuit
19	C	—	Exclude
20	C	—	Exclude
21	C	—	Exclude
22	B	Jolpica grid + lap-1 positions	Positions gained on lap 1
23	B	Jolpica /pitstops/	Pit duration consistency by team
24	B	Jolpica /status/	Mechanical DNF rate by constructor
25	A*	yfinance club shares (MANU, BVB.DE, JUVE.MI, AJAX.AS — UNVERIFIED) + hand-compiled elimination dates	Event study
26	B	StatsBomb Open Data (verified; credit + logo required)	Does goals-minus-xG persist?
27	C	—	Exclude
28	B	football-data.co.uk	Points vs rest days
29	B	StatsBomb events	Corner outcomes by delivery type
30	C	—	Exclude
31	C	—	Exclude
32	C	—	Exclude
33	A*	football-data.co.uk results + hand-compiled sackings (source per row)	Regression-to-mean controlled
34	A	football-data.co.uk CSVs: Referee, HF/AF, HY/AY, HR/AR (verified via notes.txt mirror)	Coverage varies by league-season: check
35	B	Premier League published payments (UNVERIFIED)	Top/bottom ratio
36	C	—	Exclude
37	B	Crypto price API (UNVERIFIED)	Match-day token returns
38	A	LOBSTER free samples: AMZN, AAPL, GOOG, INTC, MSFT, levels 1/5/10 (verified)	One day only: illustrative
39	C	—	Exclude

#	Tier	Dataset(s)	Note / reframe
40	A	Kaggle Give Me Some Credit / Home Credit (UNVERIFIED; competition rules)	Check terms before bundling
41	A	yfinance daily prices (UNVERIFIED)	Pre-define pair list
42	A*	Freddie Mac Single-Family Loan-Level Dataset (UNVERIFIED; registration)	Use sample files
43	B	Climate TRACE (UNVERIFIED)	Reported vs estimated emissions
44	C	—	Exclude
45	B	Artemis deal directory (UNVERIFIED)	Spread vs expected loss
46	B	FRED FX + rates (UNVERIFIED IDs)	Carry return and drawdown
47	B	FRED ICE BofA OAS (UNVERIFIED)	Spread widening in sell-offs
48	C	—	Exclude
49	B	ONS / OpenStreetMap (UNVERIFIED)	Borough rent vs amenities
50	A	FRED T10Y3M (seen), T10Y2Y, USREC (IDs UNVERIFIED) + NY Fed probabilities as benchmark	Small and clean: first case

8. Tier A build order

Domains alternate; each case adds one or two new concepts on top of earlier ones.

Order	Problem	Domain	New concepts
1	P50 Yield curve → recession	Finance	Time series, lagged predictors, logistic/probit, rare events, AUC/Brier
2	P34 Home advantage & ghost games	Football	Proportions, difference-in-differences, natural experiments, multiple testing
3	P3 Delay propagation	Aviation	Large-file aggregation, shares, decomposition, heatmaps
4	P13 Tyre degradation	F1	Regression with confounders, fixed effects, change points
5	P1 Jet fuel cross-hedge	Aviation	Stationarity (ADF), levels vs changes, hedge ratio, unit conversion
6	P33 New-manager bounce	Football	Regression to the mean, matched controls
7	P18 Safety-car rates	F1	Binomial rates, shrinkage / partial pooling
8	P41 Pairs trading	Finance	Cointegration, lookahead, backtesting, multiple testing
9	P14 Undercut	F1	Observational treatment effect, selection bias
10	P40 Credit scoring	Finance	Classification, imbalance, AUC, calibration, leakage
11	P25 UCL-exit event study	Football/Finance	Abnormal returns, event windows
12	P38 Order-book imbalance	Finance	Microstructure, high-frequency pitfalls
13	P42 Mortgage prepayment	Finance	Survival / hazard models, censoring

9. Solution blueprints (pipeline pre-solves these)

Rules for every blueprint: provenance per raw file; all numbers to expected_results.json with pytest; time-based splits for time-ordered data; each case ships a model answer, rubric (4–7 points), red flags (3–6), chart JSON + PNG. **No result numbers appear below** — the pipeline computes them.

P50 — Does 2s10s inversion predict US recessions?

- **Decision:** portfolio manager — reduce risk when the curve inverts? KPIs: 12-month-ahead recession probability, AUC, Brier score.
- **Data:** FRED T10Y2Y, T10Y3M (daily), USREC (monthly). Missing values may be '.' — confirm on download; parse with na_values='.'. Resample daily to monthly means.
- **Target:** recession at any point in t+1..t+12 (or at exactly t+12, NY Fed style). Choose one and state it.
- **Charts:** spread with shaded recessions; fitted probability over time; spread vs forward recession indicator.
- **Method:** univariate probit/logit (Estrella–Mishkin); compare 2s10s with 10y–3m; HAC standard errors (overlapping targets).
- **Validate:** expanding-window out-of-sample; AUC, Brier; compare to NY Fed published series.
- **Pitfalls:** few events; USREC is dated after the fact (not real-time); term-premium changes (Fed 2018 note).
- **Red flags:** 'inversion causes recessions'; '100% accurate'; recession months used as predictors; random split.

P34 — Does crowd pressure bias referees?

- **Decision:** league integrity analyst / betting-model owner. KPIs: home win rate, home goal difference, home-minus-away yellow cards per foul.
- **Data:** football-data.co.uk league CSVs 2017/18–2021/22; hand-compiled ghost-game periods with one source per league.
- **Cleaning:** columns vary by season; fouls missing in many league-seasons (filter per column); HY/AY count all yellows.
- **Method:** differences in proportions with vs without fans; Poisson/negative binomial for cards with team effects; Holm or BH correction across leagues.
- **Literature:** systematic review: home advantage fell in ghost games, mostly via reduced referee bias; results vary by league.
- **Pitfalls:** ghost games had congested schedules (confounder); many tests.
- **Red flags:** 'referees are corrupt'; raw card counts without foul denominator.

P3 — How much US flight delay is propagated, and where?

- **Decision:** airline network operations. KPIs: share of delay minutes that are LateAircraftDelay; propagation by hub × hour.
- **Data:** BTS On-Time monthly files (1–2 years). Cause fields only for flights 15+ min late; prorated by minutes.
- **Cleaning:** cancelled/diverted have null delays; local hhmm times; crossing midnight.
- **Charts:** stacked bar of cause shares by month; hub × hour heatmap; rolling 30-day share.
- **Pitfalls:** carrier-reported categories are not causal; cancellations remove worst cases.
- **Red flags:** 'weather causes most delays' (much weather impact is coded as NAS — verify on BTS page before teaching).

P13 — Where is the tyre degradation cliff?

- **Decision:** race strategist. KPIs: lap-time loss per lap of tyre age by compound/circuit; change point.
- **Data:** FastF1 laps (2019+); remove in/out laps, SC/VSC laps (track status), traffic laps.
- **Method:** per-stint regression of lap time on tyre age with stint fixed effect, controlling lap number (fuel burn proxy); segmented regression for the cliff; leave-one-race-out validation.
- **Pitfalls:** fuel burn-off masks degradation; teams pit before the cliff (censoring); track evolution.
- **Red flags:** 'lap times get faster so tyres improve with age'.

P1 — How should an airline cross-hedge jet fuel with crude?

- **Decision:** airline treasury. KPIs: minimum-variance hedge ratio $h^* = \rho \cdot \sigma S/\sigma F$; hedge effectiveness ($R^2$ on changes).
- **Data:** DJFUELUSGULF ($/gallon) and DCOILBRENTEU ($/barrel). Convert: 1 barrel = 42 US gallons (EIA glossary, verified).
- **Cleaning:** align trading days; drop '.' values with a logged count; treat April 2020 explicitly.
- **Charts:** both prices in same units; crack spread; scatter of weekly returns; rolling 52-week hedge ratio.
- **Method:** ADF on levels vs differences; h^* on changes not levels; optional cointegration/ECM; rolling estimation; out-of-sample variance reduction.
- **Literature:** Turner & Lim; TRF (2016) — heating oil is a better proxy than Brent/gasoil. Say so in the case.
- **Red flags:** 'R^2 0.98 in levels = perfect hedge'; mixing $/barrel and $/gallon; full-sample in-sample evaluation.

P33 — Is the new-manager bounce real?

- **Decision:** club board. KPI: change in points per game over 5–10 games before/after, vs matched control teams in a similar slump.
- **Data:** football-data.co.uk results; hand-compiled sackings CSV — pipeline refuses rows without a source URL.
- **Method:** controls from same league/season with similar PPG dip; difference-in-differences; adjust opponent strength (betting odds in same files).
- **Literature:** Erasmus release and ter Weel's study: bounce largely regression to the mean.
- **Red flags:** 'PPG rose after the sacking so it worked'; no controls.

P18 — Safety-car probability by circuit

- **KPI:** P(≥1 SC/VSC per race) per circuit. **Data:** FastF1 track status 2018+.
- **Method:** beta-binomial with partial pooling (few races per circuit); hazard by lap.
- **Pitfalls:** layout and regulation changes; small samples.
- **Red flags:** '3 of 4 races had an SC, so it's 75%'.

P41 — Pairs trading

- **Data:** daily adjusted closes for a pair list defined *before* testing (same industry).
- **Method:** Engle–Granger/Johansen on formation window; z-score rules on next window; transaction costs.
- **Pitfalls:** multiple testing; survivorship in ticker lists; lookahead in hedge ratio; restated adjusted prices.
- **Red flags:** 'high correlation means cointegrated'.

P14 — When does the undercut work?

- **Method:** for close-running pairs, gap change across the stop cycle vs tyre-age difference, compound, pit loss; exclude SC stops.
- **Pitfalls:** teams choose when to undercut (selection); traffic.
- **Red flags:** causal claims without stating selection.

P40 — Credit default scoring

- **Method:** logistic baseline then gradient boosting; AUC, KS, calibration plot; threshold chosen by expected cost.
- **Pitfalls:** leakage from post-application features; class imbalance (always predicting 'no default' already scores high accuracy — compute the base rate in the pipeline); fairness/proxies.
- **Red flags:** reporting accuracy alone.

P25 — Club shares after Champions League elimination

- **Method:** market model on pre-event window; abnormal returns over [−1, +3]; CAR with tests across events.
- **Pitfalls:** thin trading; anticipation (odds priced the loss); few events; evening match → next trading day.
- **Red flags:** attributing every daily drop to the match.

P38 — Does order-book imbalance predict the next move?

- **Method:** imbalance = (bid size − ask size)/(bid size + ask size) at level 1; logistic model of next mid-price change; test on second half of day.
- **Pitfalls:** one day only; tick size; prices stored ×10000 (verify in LOBSTER readme); dummy levels with volume 0.
- **Red flags:** 'this is a profitable strategy'.

P42 — Mortgage prepayment hazard

- **Method:** loan-month panel; discrete-time hazard (logit) with rate incentive, loan age, LTV; Kaplan–Meier curves.
- **Pitfalls:** right-censoring; default vs prepayment are competing risks; vintage effects.
- **Red flags:** treating active loans as 'never prepay'.

10. Build prompts — paste into Claude Code, one per session, in order

Before Prompt 1: create an empty folder **case-trainer**, copy this PDF to **docs/spec.pdf** and Gemini's PDF to **docs/source/**. After each prompt: test on the S21, commit to git, then continue. Start a fresh session when a session gets long.

Prompt 1 — Read the spec, plan, set up the repo

```
Read docs/spec.pdf completely. It is the single source of truth. Section 2 (Do's and Don'ts)
overrides everything, including your own assumptions.

1. Enter plan mode. Propose: repo layout (Section 4), the pipeline framework, the content
   bundle schema, the Flutter app architecture, and the grading engine design (Section 5).
   List every package you plan to use with the version you verified on pub.dev / PyPI today,
   and flag any that are discontinued. Wait for my approval.
2. After approval: create the monorepo (pipeline/, app/, docs/, sources.json), a Python
   virtual env with a lockfile, and a new Flutter project in app/ (Android only, arm64,
   minSdk 26, latest stable targetSdk).
3. Write a SHORT CLAUDE.md (under 40 lines) that summarises Section 2's rules and says
   "full spec: docs/spec.pdf". Write docs/ROADMAP.md listing Prompts 2-10.
4. git init and commit. Tell me how to install a debug build on my Galaxy S21 over USB
   (adb) and how I enable USB debugging.
Definition of done: an empty Flutter app runs on my S21; pytest and flutter test both run.
```

Prompt 2 — Pipeline framework (provenance, results, charts, content build)

```
Build the pipeline framework only. No cases yet.

1. pipeline/common/provenance.py: download helper that saves raw files untouched and writes
   provenance.json (URL, UTC timestamp, SHA-256, licence note, rows, columns) and a
   transform log (every filter/drop with counts and reason).
2. pipeline/common/results.py: ResultStore that writes expected_results.json with value,
   unit, description and the function that produced it.
3. pipeline/common/charts.py: export each chart as (a) JSON for the Flutter app:
   type (line/scatter/histogram/box/bar/stacked_bar/heatmap), series, axis titles with
   units, date axes, shaded regions, annotations, LTTB downsampling to < 2,000 points;
   and (b) a matplotlib PNG fallback styled for a phone screen (large fonts).
4. pipeline/build_content.py: validate every case (Section 4 rules: missing result keys,
   unverified sources, weights not summing to 100, missing calibration set => FAIL) and
   write app/assets/content/content.sqlite + chart files.
5. sources.json schema: key, title, authors, year, url, verified_on, quote, status.
6. Tests for all of the above.
Do not invent any case data. Use a tiny test fixture created inside the tests only.
```

Prompt 3 — Solve Case 1 (P50 Yield curve) end to end in the pipeline

```
Build pipeline/cases/p50_yield_curve following the blueprint in Section 9 exactly.

1. Verify the FRED series IDs T10Y2Y, T10Y3M and USREC by opening their FRED pages; record
   units and frequency in provenance. If any ID is wrong, stop and tell me.
2. Download, clean (log every dropped row), and analyse: charts, probit/logit with HAC
   errors, expanding-window out-of-sample predictions, AUC and Brier score.
   Compare with the NY Fed's published recession probabilities if you can download them;
   if not, say so.
3. Every number goes to expected_results.json with a pytest asserting it.
4. Write content.yaml for all 7 phases (Section 1 table): MCQs, code-reading steps using the
   REAL pipeline code and its REAL captured outputs, chart_explain steps with the 6-question
   template, result interpretation (Claim-Evidence-Reasoning), and the final memo.
   Each graded step has a rubric (4-7 points with paraphrases, keyword groups, numeric
   anchors) and red flags from Section 9. Model answers use {expected.key} placeholders.
5. Add sources: Estrella & Mishkin 1996 and NY Fed yield curve FAQ — open each, store a
   short quote under 15 words, mark VERIFIED.
6. Write 30-50 calibration answers per graded step (strong/partial/wrong/red-flag), labelled.
   Mark the ones I should personally review.
7. Run build_content.py. Show me the key results and the model answers for review.
```

Prompt 4 — Flutter app shell and case player

```
Build the Flutter app to play cases from content.sqlite. Mobile-first for a 6.2-6.7 inch
screen, dark and light theme, large touch targets (48dp+).
```

```
1. On first run, copy content.sqlite from assets; create progress.sqlite (attempts, scores,
   hints used, time, answers, review schedule).
2. Screens: Home (4 domains, case cards showing tier, concepts, progress), Case player
   (one step per screen, phase progress bar, resume exactly where I left off), Progress,
   Learn (glossary), Settings.
3. Step widgets: mcq, multi_select, order_steps, code_read (syntax-highlighted real code),
   code_predict (I type/choose the output; compare with captured output), code_fill,
   code_fix, numeric (tolerance + units), kpi_builder, chart_explain, result_explain,
   memo, concept_card. Free-text steps use a large text field with the explanation template
   shown as an optional guide.
4. Every number rendered in the UI comes from content.sqlite results — add a widget test that
   fails if any hard-coded number appears in step text.
5. Use Riverpod (or Bloc if you justify it) and go_router. Verify package versions.
Definition of done: I can play the P50 case on my S21 up to the first chart (grading can be a
stub returning 'not graded yet').
```

Prompt 5 — Interactive charts

```
Render the pipeline's chart JSON as interactive charts.

1. Use Syncfusion Flutter Charts (I will register for the free Community Licence; tell me
   where to put the key) with fl_chart as fallback. Support every chart type in the JSON
   schema, date axes, shaded regions (recessions), annotations, a fitted line on scatter.
2. Interactions: pinch-zoom and pan, tap a point for a tooltip with units, toggle series in
   the legend, a 'reset view' button, and a full-screen landscape mode.
3. Show the PNG fallback if a chart fails to parse, and log the error.
4. Performance on my S21: measure frame times while zooming the largest chart; target smooth
   scrolling (report the numbers you measured, do not estimate).
```

Prompt 6 — The examiner (grading engine, 3 attempts, 85%)

```
Implement Section 5 of the spec exactly.

1. Embedding model: all-MiniLM-L6-v2 ONNX (try the arm64 int8 file first) via ONNX Runtime.
   Verify the Flutter plugin on pub.dev and that it supports Android arm64 and 16 KB pages.
   Implement the WordPiece tokenizer with the model's vocab.txt and unit-test it against
   Python tokenizer outputs (generate the expected token IDs in the pipeline).
   If ONNX integration fails, fall back to MediaPipe Text Embedder via a platform channel.
2. Scoring: sentence split, max-cosine per rubric point mapped by the fitted logistic curve,
   keyword groups with synonyms, numeric anchor extraction with tolerance, red flags with a
   negation check, weighted total, pass = >= 85 and no critical red flag.
3. Attempt loop: attempt 1 -> missing areas + template + analysis pointer; attempt 2 -> targeted
   hints + red-flag explanations + which analysis to redo; attempt 3 -> model answer with
   highlighted rubric points linked to chart regions/numbers, then schedule spaced re-test
   (1, 3, 7, 21 days).
4. Grading must run fully offline. Measure on my S21: time to load the model, time to grade
   a 120-word answer. Report measured numbers.
```

Prompt 7 — Calibrate and test the grader

```
1. In the pipeline, compute the same embeddings in Python and fit the logistic mapping and
   weights per step using the labelled calibration answers. Export parameters into
   content.sqlite.
2. Report per step: false-pass rate, false-fail rate, Cohen's kappa vs my labels. Target
   false-pass < 5%. Show me the worst disagreements so I can relabel or improve the rubric.
3. Add a Dart test that grades every calibration answer on-device logic and matches the
   Python scores within a small tolerance.
4. Add a 'Disagree with grade' button in the app that saves my answer + my label into an
   export file I can feed back into calibration.
```

Prompt 8 — Build the remaining Tier A cases (repeat in batches)

```
Using P50 as the template and the build order in Section 8, build the next batch:
[PASTE e.g. P34, P3, P13].

For each case: follow its Section 9 blueprint; verify every dataset ID/URL/column before use;
provenance + expected_results + pytest; real cleaning issues shown in the Data phase;
charts with 6-question explain steps; method + validation; limitations step; memo;
calibration set; sources opened and marked VERIFIED.
For A* cases, hand-compiled tables need a source URL on every row (pipeline rejects rows
without one).
QUALITY GATE before marking done: a fresh subagent that has not seen your work plays the case
as a demanding data science lecturer and reviews statistics, charts, rubrics and causal
```

```
claims; fix everything it finds. Commit each case separately and give me a review list.
If real data for a case cannot be obtained, STOP and report - never substitute synthetic data.
```

Prompt 9 — Learn tab, spaced review, progress, portfolio export

```
1. Glossary cards for every concept used (from the cases): plain meaning, one-line formula,
   'how it fools people', which cases use it. Terms in step text are tappable.
2. Daily 5-minute warm-up: spaced-review items + 'read one chart' drills from completed cases,
   weighted toward my most frequent red flags.
3. Progress: cases per domain, pass rate per attempt, top 5 recurring misconceptions.
4. Portfolio export per completed case: the pipeline already produces a runnable Jupyter
   notebook of the solution; the app exports MY explanations + memo as Markdown, and shares
   both via Android share sheet so I can send them to my PC. Include a README draft with only
   verified sources and only pipeline numbers, plus 2-3 CV bullets I must rewrite myself.
```

Prompt 10 — Optional AI feedback and release build

```
1. Optional local LLM (off by default): Gemma 3 1B int4 via the current maintained Flutter
   package (check flutter_gemma vs its replacement). Only rephrases the deterministic feedback
   into friendlier text, max ~150 tokens, never changes score or numbers. Model stored in
   app storage, loaded on demand. Measure RAM, tokens/sec and temperature on my S21 and
   report the measured values; disable it if it makes the phone hot or slow.
2. Optional online feedback via Claude API with my own key (stored on device only).
3. Release build: flutter build apk --release --split-per-abi; install the arm64 APK via adb.
   Report APK size and total installed size.
4. Final checks on the S21: airplane-mode full case run, cold start time, battery use for a
   20-minute session. Write docs/HOW-TO-USE.md.
```

11. Verification register

Verified in research (still re-open before shipping)

- FRED DJFUELUSGULF and DCOILBRENTEU pages; EIA: 1 barrel = 42 US gallons
- BTS On-Time Performance database info and delay-cause page
- FastF1 docs and repo; Jolpica-F1 docs; StatsBomb open-data terms; LOBSTER samples and report
- football-data.co.uk column notes (via GitHub mirror of notes.txt)
- Estrella & Mishkin (1996); NY Fed yield curve FAQ; FEDS Note 2018; Wright (2006)
- Turner & Lim (2015); TRF (2016); ghost-game systematic review; Leitner & Richlan (2021); Erasmus release
- López de Prado (2018) Wiley; Dunlosky et al. (2013)
- Syncfusion Community Licence terms; serious_python (pub.dev); MediaPipe Text Embedder; all-MiniLM-L6-v2 files; flutter_gemma discontinuation notice
- Anthropic docs: Claude Code best practices; reduce hallucinations

UNVERIFIED — must be checked before use

- FRED IDs T10Y2Y and USREC; FRED '.' missing-value convention in current CSV exports
- yfinance tickers and Yahoo terms; Kaggle dataset rules; Freddie Mac dataset terms
- fl_chart licence; drift/sqflite/Riverpod/go_router versions; ONNX Runtime Flutter plugin choice
- Any performance figure on the Galaxy S21 (none published — measure on the device)
- All 'None verified' reference cells in Section 6

12. Sources

- Turner & Lim (2015) Hedging jet fuel price risk — pmc.ncbi.nlm.nih.gov/articles/PMC7147842/
- Transportation Research Forum (2016) Airline Fuel Hedging — trforum.org/wp-content/uploads/2016/10/2016v55n1_03_AirlineFuelHedging.pdf
- FRED DJFUELUSGULF — fred.stlouisfed.org/series/DJFUELUSGULF
- FRED DCOILBRENTEU — fred.stlouisfed.org/series/DCOILBRENTEU
- FRED T10Y3M — fred.stlouisfed.org/series/T10Y3M
- EIA Glossary: barrel — eia.gov/tools/glossary/index.php?id=barrel

- BTS delay causes — transtats.bts.gov/ot_delay/ot_delaycause1.asp
- BTS On-Time database — transtats.bts.gov/DatabaseInfo.asp?QO_VQ=EFD
- FastF1 — github.com/theOehrly/Fast-F1 · docs.fastf1.dev/core.html
- Jolpica-F1 — github.com/jolpica/jolpica-f1/blob/main/docs/README.md
- StatsBomb Open Data — github.com/statsbomb/open-data
- football-data.co.uk notes (mirror) — github.com/kokosing/autobet/blob/master/data/www.football-data.co.uk/notes.txt
- LOBSTER samples — data.lobsterdata.com/info/DataSamples.php · report: data.lobsterdata.com/info/docs/LobsterReport.pdf
- Estrella & Mishkin (1996) — newyorkfed.org/research/current_issues/ci2-7.html
- NY Fed yield curve FAQ — newyorkfed.org/research/capital_markets/ycfaq
- FEDS Note (2018) — federalreserve.gov/econres/notes/feds-notes/predicting-recession-probabilities-using-the-slope-of-the-yield-curve-20180301.html
- Wright (2006) FEDS 2006-07 — federalreserve.gov/pubs/feds/2006/200607/index.html
- Ghost games systematic review — pmc.ncbi.nlm.nih.gov/articles/PMC8724651/
- Leitner & Richlan (2021) — frontiersin.org/journals/sports-and-active-living/articles/10.3389/fspor.2021.720488/full
- Erasmus Univ. Rotterdam — eur.nl/en/news/irrational-managerial-changes-remain-norm-europes-top-football-leagues
- Pinnacle (ter Weel summary) — pinnacle.com/betting-resources/en/Soccer/the-myth-of-the-new-manager-effect/BAZ2FQLS2QCBJ795
- López de Prado (2018) — wiley.com/en-us/Advances+in+Financial+Machine+Learning-p-9781119482086
- Dunlosky et al. (2013) — whz.de/fileadmin/lehre/hochschuldidaktik/docs/dunloskiimprovingstudentlearning.pdf
- Syncfusion Community Licence — syncfusion.com/products/communitylicense
- serious_python — pub.dev/packages/serious_python
- flutter_gemma — pub.dev/packages/flutter_gemma
- MediaPipe Text Embedder — ai.google.dev/edge/mediapipe/solutions/text/text_embedder
- all-MiniLM-L6-v2 — huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Gemma 3 1B LiteRT — huggingface.co/litert-community/Gemma3-1B-IT
- Claude Code best practices — code.claude.com/docs/en/best-practices
- Reduce hallucinations — docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails/reduce-hallucinations